KLAi

SVD and PCA: rotate, stretch, rotate, and keep only what matters

How can a huge matrix be squeezed into a few numbers, and how does LoRA fine-tune a large model cheaply?

Mathematics for AI and Machine Learning | Session 6 of 18 | Linear algebra

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Describe the singular value decomposition (SVD) as rotate, stretch, rotate, and read its singular values

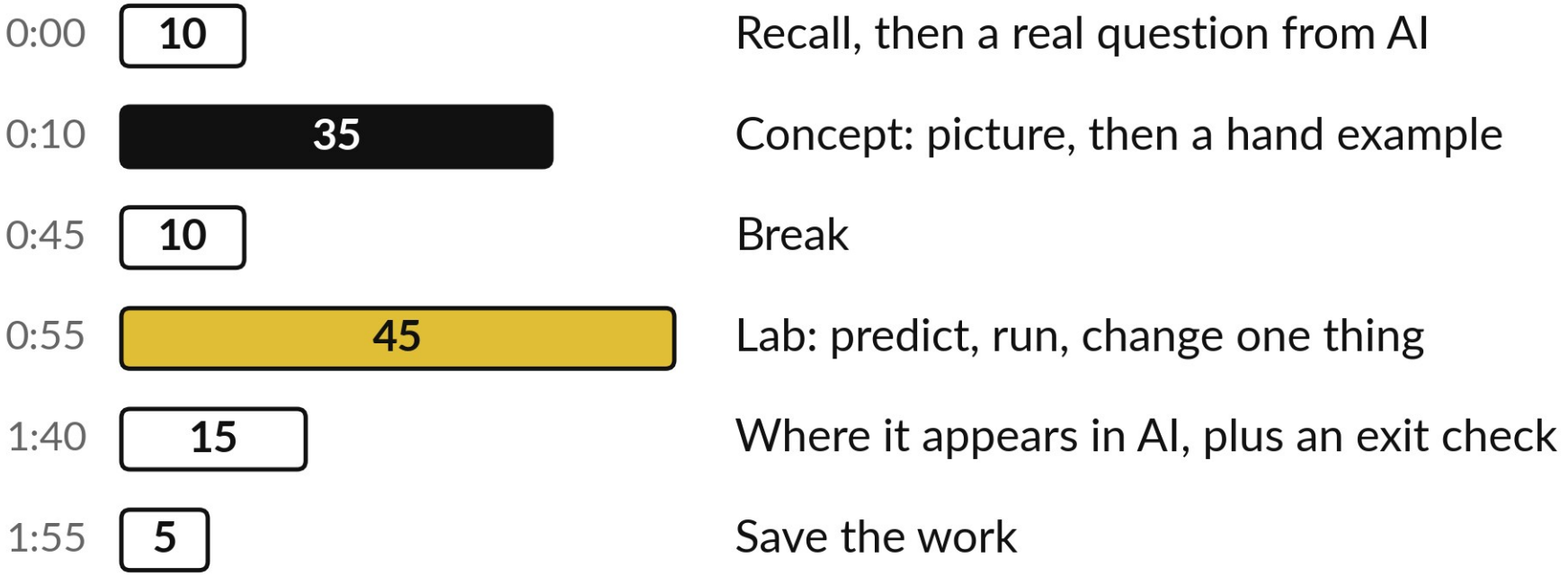


Build a low-rank approximation of an 8 by 8 image, and choose how many components to keep



Do principal component analysis (PCA) by hand: centre, covariance, top direction, projection and explained variance

Your two hours: break a matrix apart, then keep the best terms



Pictures first. Then the lab in Python or in a spreadsheet. Then where it shows up in AI.

Three questions from last time

1. The covariance matrix of two marks has eigenvalues 102 and 2. What do these two numbers mean?

2. In power iteration, why divide by the largest entry, and what does that entry estimate?

3. How are the trace and the determinant of a 2 by 2 matrix linked to its eigenvalues?



Answer aloud first. Then we check together.

Answers to the three questions from Session 5

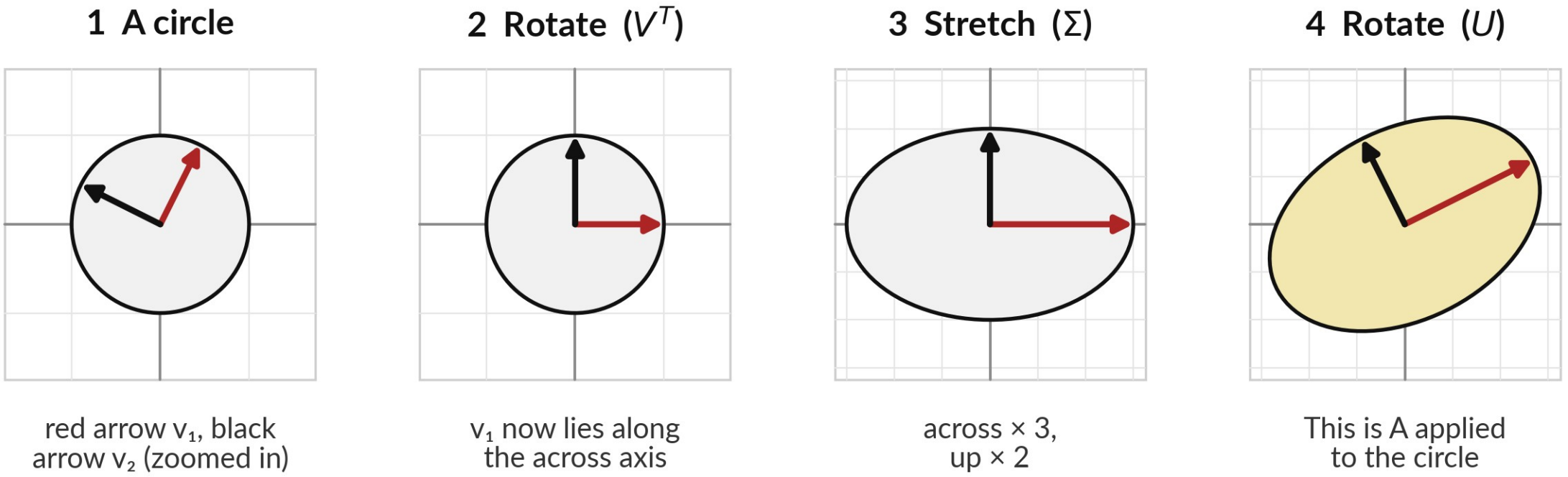
Question	Short answer
The covariance matrix of two marks has eigenvalues 102 and 2. What do these two numbers mean?	Each eigenvalue is the variance of the data along its eigenvector. Almost all the spread lies along the first direction.
In power iteration, why divide by the largest entry, and what does that entry estimate?	Dividing keeps the numbers from growing. The largest entry estimates the top eigenvalue.
How are the trace and the determinant of a 2 by 2 matrix linked to its eigenvalues?	The eigenvalues add up to the trace and multiply to the determinant.

If one surprised you, look back at Session 5 before the lab. Eigenvalues return today.

Words we use today

	Word	What it means
	**Singular value decomposition (SVD)**	Any matrix written as rotate, stretch, rotate: $A = U\Sigma V^T$.
	**Singular value**	A stretch factor in Σ. The singular values are listed from largest to smallest.
	**Low-rank approximation**	A matrix rebuilt from only its first few singular values. It needs far fewer numbers to store.
	**Principal component analysis (PCA)**	Finding the directions of greatest spread in data, then keeping only the first few.
	**LoRA (low-rank adaptation)**	A published way to fine-tune a large model by training two thin matrices.

Any matrix acts as rotate, stretch, rotate



v_1 and v_2 are the right singular vectors: the two directions that A turns onto the axes.

A has rows (2, 2) and (−1, 2). Rotate the arrows onto the axes, stretch by 3 and 2, rotate again.

The singular value decomposition writes A as $U\Sigma V^T$

Definition

$$A = U \Sigma V^T$$

Singular values of our A

$$\Sigma = \mathrm{diag}(3,\ 2)$$

Symbol	What it means	In our data
V^T	First rotation. Its rows are the right singular vectors, unit arrows in the input space.	rows (0.447, 0.894) and (0.894, −0.447)
Σ	The stretch: diagonal matrix of singular values, largest first. Not the sum symbol of Session 1.	3 and 2
U	Last rotation. Its columns are the left singular vectors, unit arrows in the output space.	columns (0.894, 0.447) and (0.447, −0.894)
Signs	A singular vector can flip sign with its partner. Tools differ.	both are correct

A can have any shape, m by n. U is m by m, Σ is m by n and V is n by n.

Worked by hand: singular values from A^TA

A has rows (2, 2) and (−1, 2). The singular values are the square roots of the eigenvalues of A^TA.

Form A^TA	Rows (5, 2) and (2, 8). It is symmetric.
Eigenvalues	T = 13 and D = 36. The root of $13^2 - 4 \times 36$ is 5. So $\lambda = 9$ and 4.
Singular values	σ_1 = the square root of 9 = 3 and σ_2 = the square root of 4 = 2.
Vectors	For $\lambda = 9$, v_1 = (1, 2) / the square root of 5 = (0.447, 0.894). Then $u_1 = Av_1 / 3$ = (0.894, 0.447).

Eigenvalue decomposition and SVD answer related questions



Eigenvalue decomposition

Square matrices only

Finds directions that only stretch

Eigenvalues can be negative or not real numbers



Singular value decomposition

Any shape, m by n

Rotate, stretch, rotate

Singular values are never negative

For a covariance matrix (symmetric, no negative eigenvalues) the two give the same numbers.

A grey-scale image is a matrix of numbers, and noise spoils its pattern

Clean pattern

1	8	1	1	1	1	8	1
1	8	1	1	1	1	8	1
1	8	1	1	1	1	8	1
1	8	8	8	8	8	8	1
1	8	8	8	8	8	8	1
1	8	1	1	1	1	8	1
1	8	1	1	1	1	8	1
1	8	1	1	1	1	8	1

an H shape: cells 1 or 8

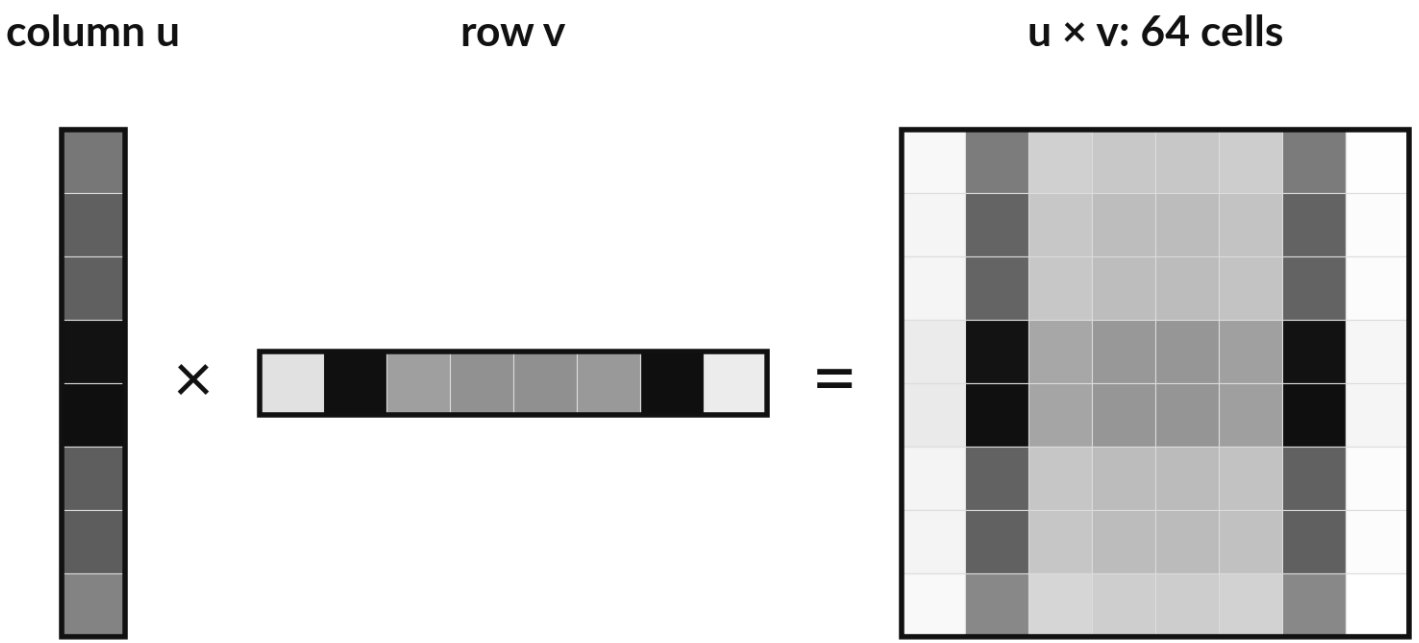
Noisy image

3	8	0	1	2	0	7	0
2	8	0	2	3	0	9	0
2	9	1	2	0	2	8	2
0	7	9	7	7	7	9	1
0	8	7	9	8	9	7	0
0	9	2	0	2	0	9	1
2	9	0	3	3	2	7	1
0	6	1	0	0	2	8	0

error 8.19 from the clean one

Illustrative: each cell is 0 (white) to 9 (black). Error = √(sum of the 64 squared cell differences).

A column times a row makes a rank 1 pattern from only 16 numbers



made from only 16 numbers (8 in u and 8 in v)

A column u of 8 numbers times a row v of 8 numbers. Cell (i, j) is the i-th number of u times the j-th number of v.

Every row of the result is a multiple of the row v. That is rank 1.

Dark cells appear where both numbers are large.

SVD adds up rank 1 terms, and a low-rank approximation keeps the first few

Sum of terms

$$A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \cdots$$

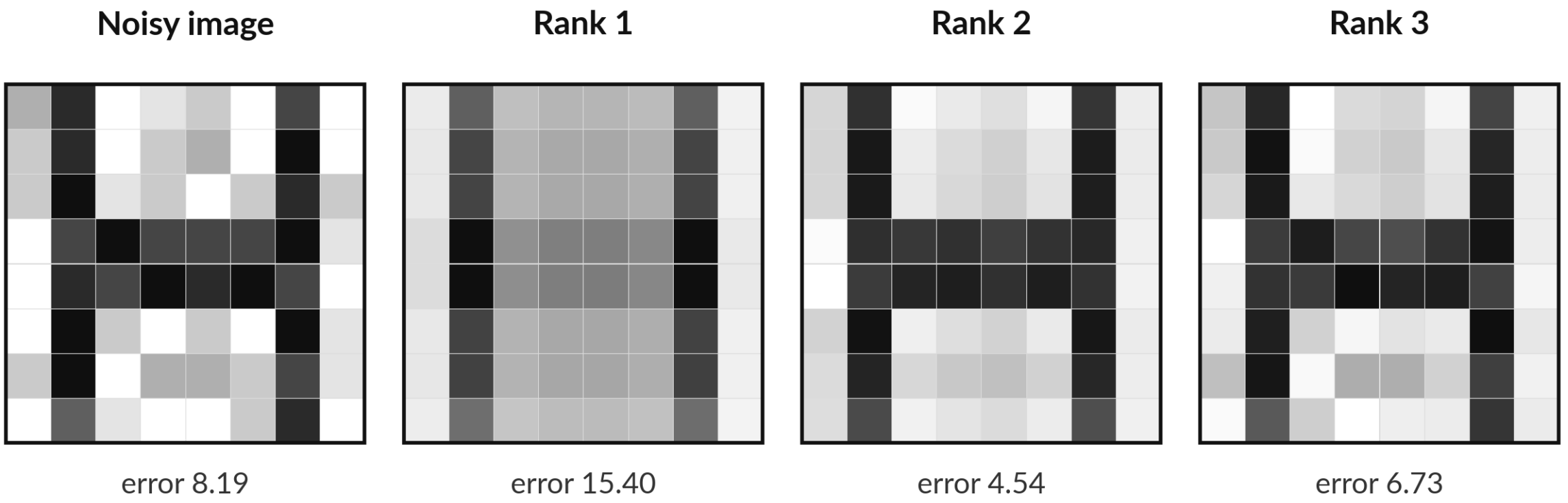
Rank k approximation

$$A_k = \sum_{i=1}^{k} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^T$$

Symbol	What it means	In our data
σ_i	The i-th singular value. Big ones carry the main pattern.	36.8, 15.1, 4.9, ...
$\mathbf{u}_i$, $\mathbf{v}_i$	The i-th left and right singular vectors, columns of U and V.	one pair per term
k	How many terms we keep. Rank k means k independent patterns.	k = 2
Storage	A rank k approximation needs k × (m + n + 1) numbers: u, v and σ for each term.	k = 2: 2 × (8 + 8 + 1) = 34 against 64

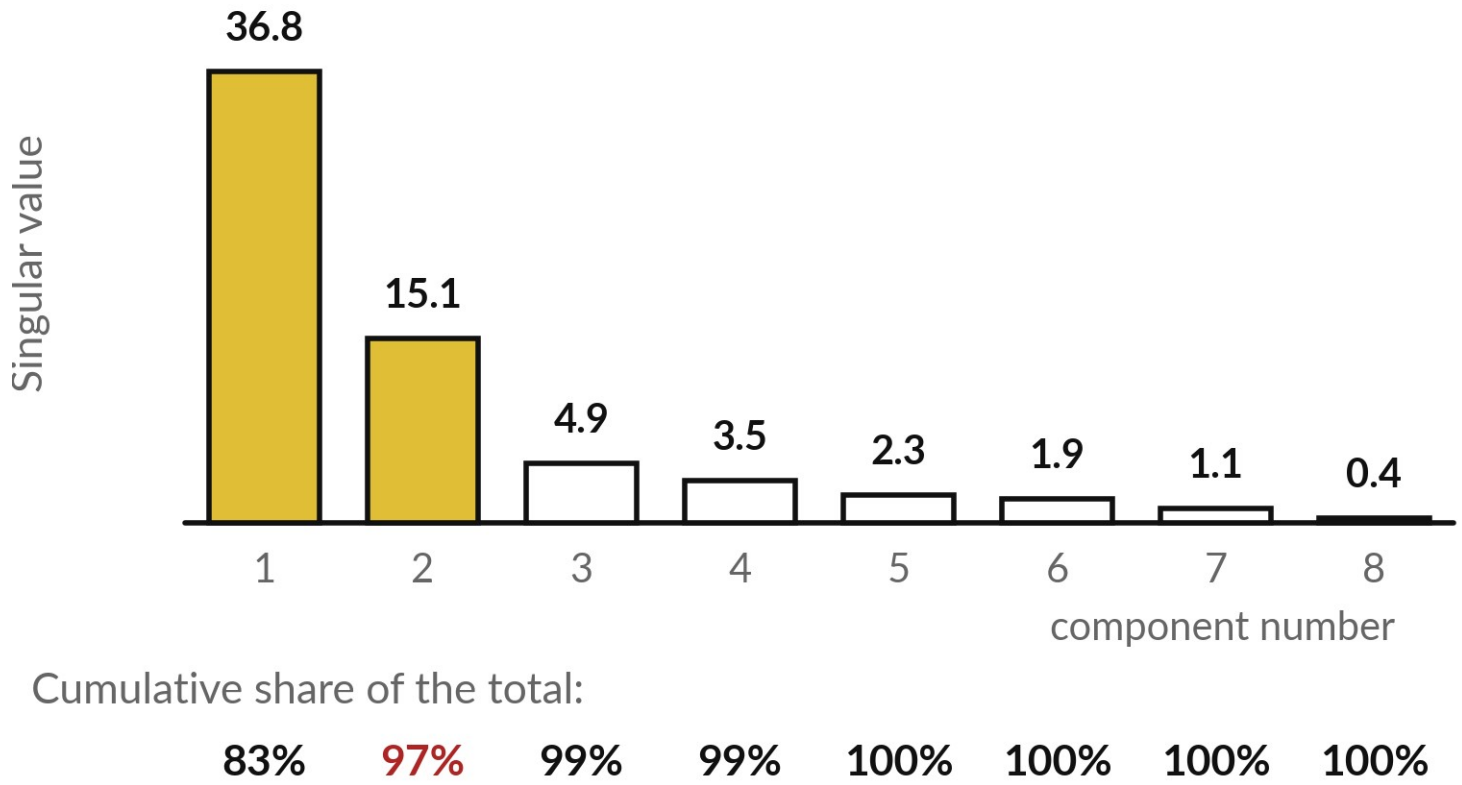
The first k terms give the closest rank k matrix to A (the Eckart-Young theorem).

Rank 2 removes most of the noise, and more terms bring the noise back



Error is measured against the clean H. Rank 2 lands closest; rank 3 starts to copy the noise.

Singular values show how many terms matter: a few big bars, then a flat tail



The 8 singular values of the noisy image, largest first. The cumulative share is the running total of σ^2 divided by the total.

Two terms hold 97 percent of the total.

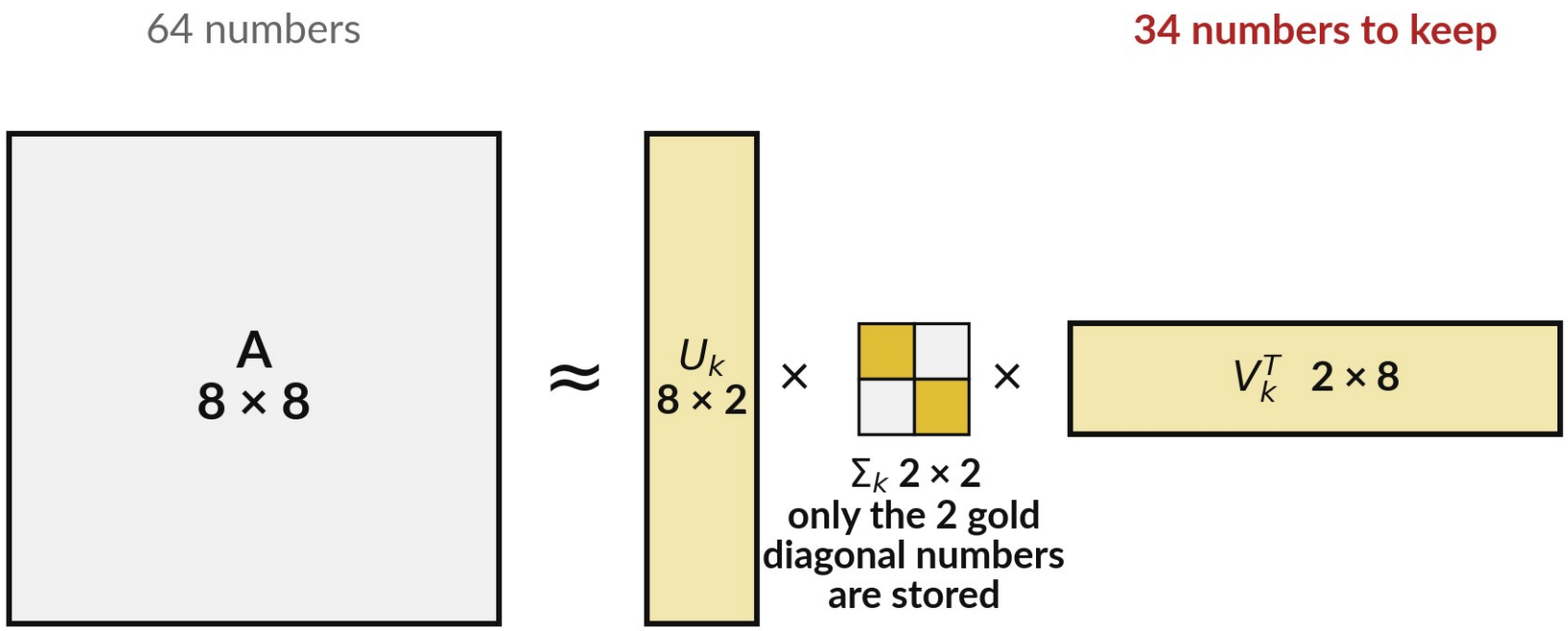
The pattern lives in the big values. Random noise spreads thinly over all eight.

Choosing the rank: error, storage and the singular values together

Rank k	Error from the clean image	Numbers to store	Share of the original 64
1	15.40	17	27%
2	4.54	34	53%
3	6.73	51	80%

In practice the clean image is unknown. Two rules: keep enough terms to reach a cumulative share such as 95 percent (here k = 2), or cut at the elbow where the bars flatten.

A low-rank approximation stores thin matrices instead of the whole matrix



Here 34 numbers replace 64. At 1,000 by 1,000 and rank 20: 40,020 against 1,000,000 (illustrative).

To find σ_1 without an SVD function, run power iteration on A^TA

Power iteration on M

$$\mathbf{g}_{k+1} = \frac{M\mathbf{g}_k}{\max|M\mathbf{g}_k|} \qquad M = A^TA$$

What it finds

$$\max|M\mathbf{g}_k| \to \sigma_1^2 \qquad \sigma_1 = \sqrt{1357.6} = 36.85$$

Symbol	What it means	In our data
M	A^TA for the noisy image A. It is 8 by 8 and symmetric, so Session 5 applies.	8 by 8
g_k	The guess after k multiplications. It starts at g_0 = (1, 1, ..., 1).	10 steps
σ_1^2	The top eigenvalue of M: the largest entry before dividing.	1357.6
v_1, u_1	v_1 is the final guess at length 1. Then $u_1 = Av_1 / \sigma_1$.	rank 1 term $\sigma_1 u_1 v_1^T$

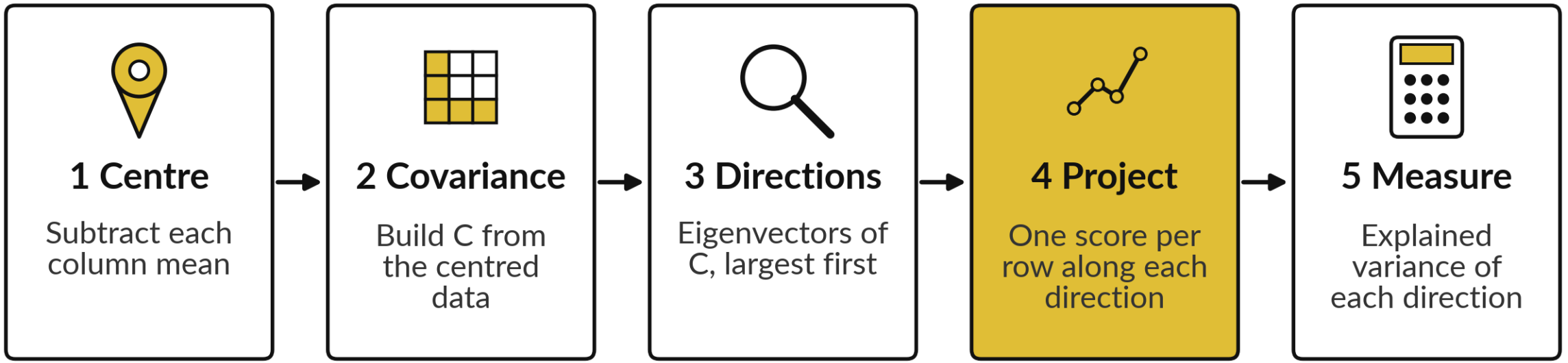
Session 5 showed that the largest entry tends to the top eigenvalue. For A^TA that eigenvalue is σ_1^2.

Deflation: subtract the rank 1 term, then repeat for σ_2 and σ_3

$M_2 = M - \sigma_1^2 v_1 v_1^T$ has the same eigenvectors as M, but its top eigenvalue is removed.

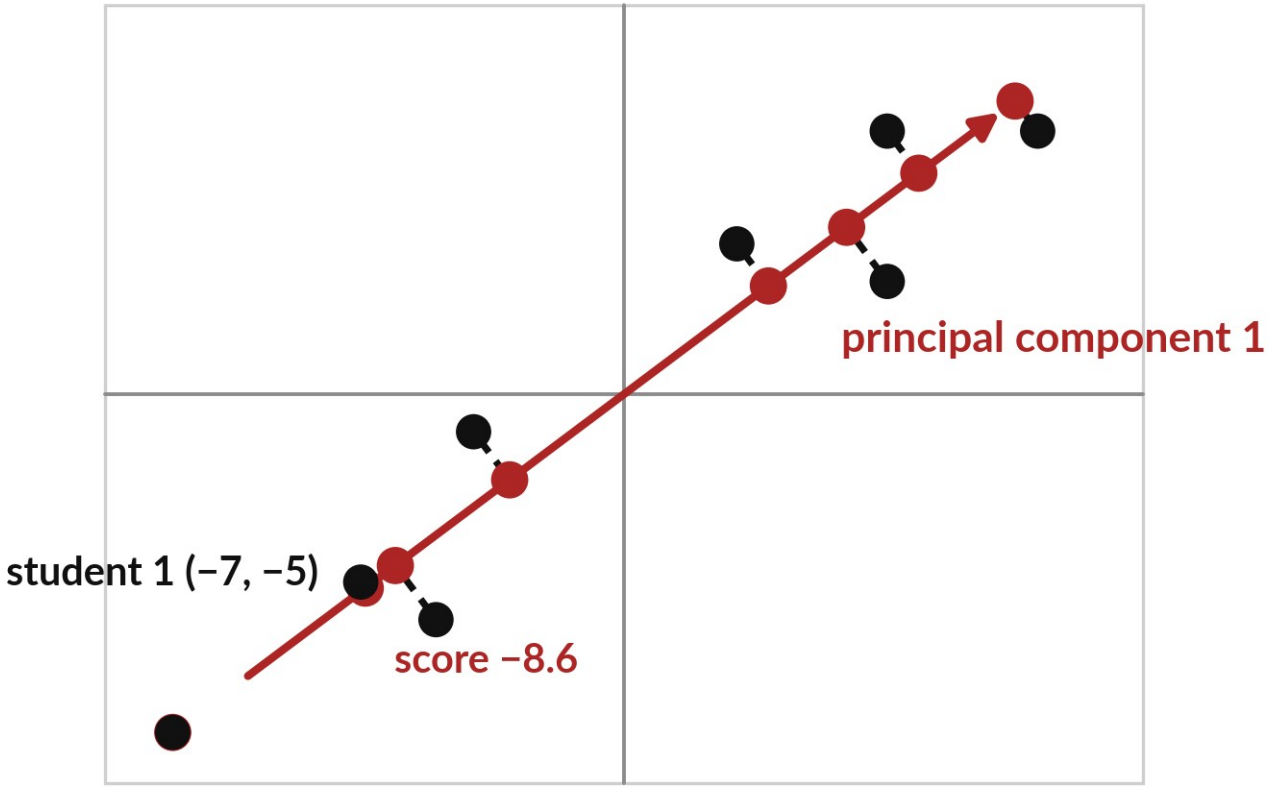
Round 2	Power iteration on M_2 gives $\sigma_2^2 = 227.1$, so $\sigma_2 = 15.07$. The second rank 1 term is $\sigma_2 u_2 v_2^T$.
Round 3	Subtract again. The next round gives $\sigma_3^2 = 23.9$, so $\sigma_3 = 4.89$.
Add the terms	A rank k approximation is the sum of the first k terms. Error from the clean image: 15.40, 4.54, 6.73 for ranks 1, 2 and 3.

PCA is five steps: centre, covariance, directions, project, measure



Principal component analysis (PCA). The directions are the principal components. Steps 1 to 3 are Session 5.

Projecting onto the first principal component turns 2 numbers per student into 1



Centred maths mark (across) and centred physics mark (up)

The eight centred marks from Session 5. Each dot drops straight onto the red arrow, the first principal component.

The red dots on the arrow are the projections.

A score is the position of the red dot along the arrow, in mark points.

The dashed lines show what is lost.

A score is a dot product with the principal component, and explained variance is a ratio

Score of student i

$$z_i = \mathbf{v}_1 \cdot \mathbf{x}_i = v_{11}\, m_i + v_{12}\, p_i$$

Explained variance ratio

$$\text{ratio}_1 = \lambda_1 \,/\, (\lambda_1 + \lambda_2)$$

Symbol	What it means	In our data
$\mathbf{v_1}$	Principal component 1: the top eigenvector of C, length 1. Its entries are v_{11} and v_{12}.	(0.8, 0.6)
$\mathbf{x_i}$	Centred marks of student i: maths m_i and physics p_i.	first student: (−7, −5)
$\mathbf{z_i}$	The score: the dot product of v_1 and x_i.	0.8 × (−7) + 0.6 × (−5) = −8.6
$\boldsymbol{\lambda}$	Eigenvalues of C: the variances along each direction.	102 and 2
Ratio	Share of the variance kept by component 1.	102 / 104 = 98.1 percent

Keep the first component and drop the second: 98.1 percent of the variance is kept, 1.9 percent is lost.

Worked by hand: the eight scores, and the variance they keep

Student	Centred maths	Centred physics	Score = 0.8 × maths + 0.6 × physics
1	−7	−5	−8.6
2	−5	−6	−7.6
3	3	4	4.8
4	7	7	9.8
5	−12	−9	−15.0
6	7	3	7.4
7	−4	−1	−3.8
8	11	7	13.0

The scores have mean 0 and variance 102 (sum of squares 714, divided by 7). The squared distances dropped add up to 14, which is 7 × 2.

PCA is the SVD of the centred data

SVD of the centred data $$X_c = U \Sigma V^T$$

Variance along component i $$\lambda_i = \sigma_i^2 \; / \; (n - 1)$$

Symbol	What it means	In our data
X_c	The centred data, one row per student and one column per mark.	8 by 2
V	Its columns are the principal components, the same as the eigenvectors of C.	v_1 = (0.8, 0.6)
σ_i	The singular values of X_c.	26.72 and 3.74
λ_i	Variance along component i: σ^2 over n − 1.	102 and 2
Scores	The columns of UΣ are the scores of all rows at once.	first score −8.6

Libraries use the SVD because it is more accurate than forming C first. Component signs may differ.

How many components to keep? Three rules, and the questions to ask

	Rule	How it works	Watch out
	**Cumulative explained variance**	Add the ratios until you reach a target such as 95 percent.	The target is a choice. Ask what the next step needs.
	**Elbow of the scree plot**	Cut where the bars stop falling steeply.	Some plots have no clear elbow.
	**Test the real task**	Try k = 1, 2, 3 and score the model on held-out data: rows kept aside, never used for fitting.	Needs the full pipeline, but is the most honest.

Common mistake: four things to remember about SVD and PCA

Mistake	What is true	What to do
Running PCA on data that is not centred	The first direction then points at the mean, not at the spread.	Subtract the column means first.
Mixing columns on different scales	A column in thousands dominates a column in units.	Use z-scores when units differ: subtract the column mean, divide by its spread (Session 14).
Treating a component as a named feature	It is a mix of all columns, and its sign is arbitrary.	Read the weights, and expect sign flips.
Keeping the most terms that fit	Small singular values often hold only noise.	Choose k with a rule, and check on held-out data.

Read the recipe before you write it: SVD, low rank and PCA

```
U, S, Vt = np.linalg.svd(M)
M2 = (U[:, :2] * S[:2]) @ Vt[:2]
Xc = X - X.mean(axis=0)
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
scores = Xc @ Vt[0]
ratio = S ** 2 / np.sum(S ** 2)
```

In plain words

U, S, Vt = ... svd of any matrix M, such as the noisy image. S is a list, largest first.

M2 = ... scales the first 2 columns of U by their singular values, then multiplies by the first 2 rows of Vt.

Xc = ... centres X, one row per student. Then PCA is the SVD of Xc.

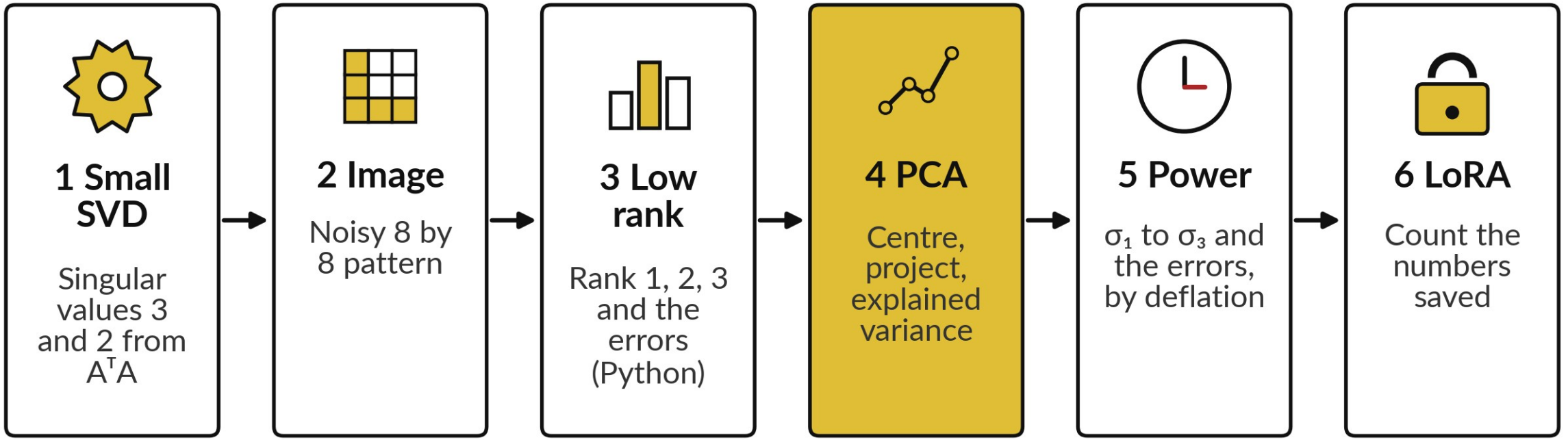
scores = ... dots each row with component 1. ratio = ... is the explained variance.



Part 2: the lab

Close the slides. Open the notebook or the workbook.

The lab has six steps, in the notebook and in the workbook



Python uses np.linalg.svd. The spreadsheet uses MMULT, SUMPRODUCT, VAR.S and rows of cells. Gold cells are yours.

Check your numbers against these before you move on

What you compute	You should see
Singular values of A with rows (2, 2) and (−1, 2)	3 and 2
First three singular values of the noisy image	36.85, 15.07 and 4.89
Error from the clean image, ranks 1, 2 and 3	15.40, 4.54 and 6.73
Eigenvalues of C for the eight marks	102 and 2
Score of the first student	−8.6
Variance of the eight scores, explained variance ratio	102 and 0.981
Largest entry after 10 power steps on A^TA (σ_1^2)	1357.6
LoRA numbers for d = 1,000 and r = 8	16,000 instead of 1,000,000

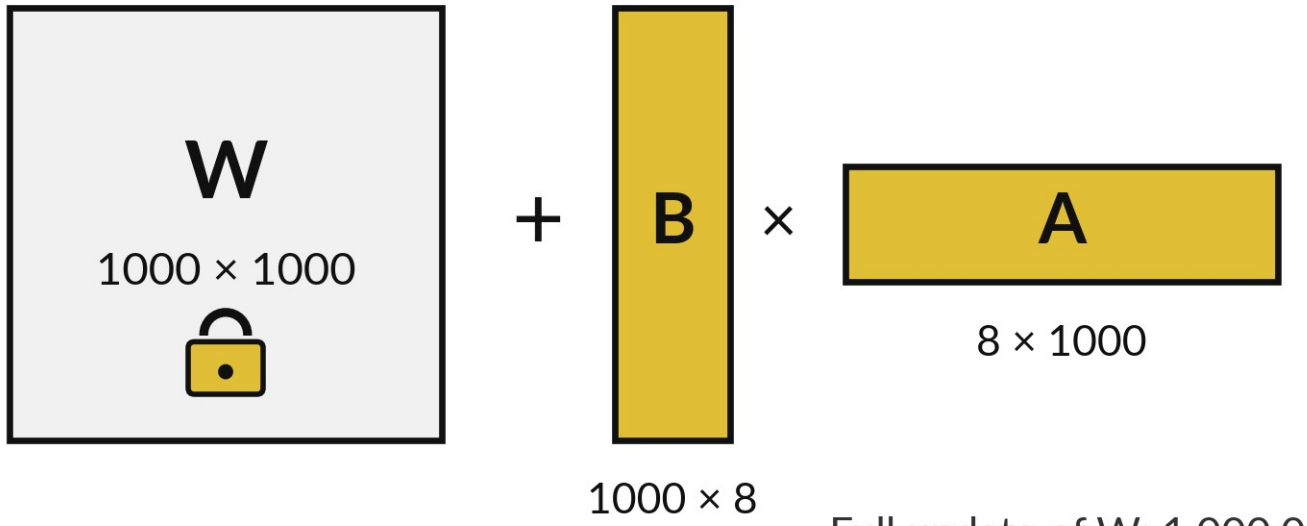
If a number differs, stop and find the step that went wrong. Signs of singular vectors may differ between tools.

Four errors you will meet, what they say and what to do

Error or symptom	What it says	What to do
S is a plain list of numbers	np.linalg.svd returns the singular values as a 1-D array.	Use np.diag(S), or scale the columns of U as in the recipe.
Low-rank image looks the same as the noisy one	k is too large, or the slice is wrong.	Slice the first k columns of U and the first k rows of Vt.
The first component points at the mean	The data were not centred.	Subtract the column means first.
#VALUE! in an array formula	The result range has the wrong size, or Enter was used.	Select the exact range and confirm as an array formula.

Where this appears in AI: LoRA trains two thin matrices in place of one huge one

Frozen (not trained) **Trained** (illustrative sizes: d = 1000, r = 8)



Full update of W: 1,000,000 numbers. B and A together: 16,000 numbers.

Illustrative sizes. LoRA (Hu et al., 2021) freezes W and trains the change as B times A.

Where this appears in AI: LoRA in numbers

Updated weights $$W' = W + B\,A$$

Trainable numbers $$2\,d\,r \quad \text{against} \quad d^2$$

Symbol	What it means	In our data
W	The pre-trained weight matrix, d by d. Frozen.	d = 1,000 (illustrative)
B and A	Thin matrices, d by r and r by d. Trained.	r = 8
Count	Numbers trained with LoRA against a full update.	2 × 1,000 × 8 = 16,000 against 1,000,000
Share	The fraction that must be trained.	1.6 percent
Start	The paper starts B at zero, so B A = 0 at the start.	published detail

Open models such as Llama can be fine-tuned this way. We make no claim about closed products.

Where this appears in AI: compression, noise removal and maps of embeddings



Low-rank approximation compresses a matrix and removes noise, as with our image.



PCA gives two or three numbers per embedding, so we can draw a map of them (Session 5).



Published research reports that low-rank updates can fine-tune large models well (Hu et al., 2021).



Closed products do not publish their weights, so we cannot show their ranks.

Where this appears in AI: check an assistant with your numbers

8 minutes

Hands-on, with any AI assistant

Ask an assistant for an SVD, then check it.

Ask for the singular values of the matrix with rows (2, 2) and (−1, 2). Expect 3 and 2.

Ask how many numbers LoRA trains for a 1,000 by 1,000 matrix with r = 8. Expect 16,000.

Ask it to show its working, and read for mistakes. Never paste confidential data.

You keep

A note of which answers matched, and where the assistant slipped.

Exit check: three questions with short answers

Question	Short answer
What are the three moves of the SVD?	Rotate (V^T), stretch (Σ, the singular values), rotate again (U).
How do you choose how many components to keep?	Use the cumulative explained variance, the elbow of the scree plot, or a test on the real task.
Why does LoRA train fewer numbers than a full update?	It trains two thin matrices B (d by r) and A (r by d) with small r, instead of one d by d matrix.

Write your own answer first, then compare. Your lab checks are part of the exit check.

Save the lab and keep the key singular values

What you keep from today

Keep the saved lab and these numbers: singular values 3 and 2 for the small matrix, 36.8 and 15.1 for the noisy image, and 98.1 percent of the variance of the eight marks on one direction.

Our question today

How can a huge matrix be squeezed into a few numbers, and how does LoRA fine-tune a large model cheaply?

Next session

Session 7: derivatives. We leave linear algebra and start calculus, the tool that tells a model which way to improve.

Take away

Saved lab
Notebook or workbook, saved

Small matrix
Singular values 3 and 2

Eight marks
98.1% on one direction

Next session
Derivatives